

LLARS – Script Creator Library

Welcome to the LLARS copy/paste reference for script creators.

This document is a companion to the **LLARS – Script Creator & Framework Guide**. The framework guide explains *why* LLARS is structured the way it is. This library focuses on *what to type*.

The examples below are intentionally written in the same **AutoHotkey v1.1.37.02** style used throughout LLARS. They are based on patterns used across the current LLARS scripts and the six script creator templates.

Use these as building blocks. Change section names, variable names, coordinates, hotkeys, timers, and actions to match the script you are creating.

Important: copy the smallest useful pattern. Do not copy an entire old script just to obtain one click, timer, or IniRead.

Table of Contents

1. How to Use This Library
 2. Config.ini Building Blocks
 3. IniRead Examples
 4. Random Sleep Examples
 5. Coordinate and Click Examples
 6. Hotkey and Send Examples
 7. Color and Pixel Examples
 8. Optional Feature Examples
 9. Timer Examples
 10. Logging Examples
 11. RunCount – Basic Loop Recipe
 12. RunCount – Advanced Loop Recipe
 13. Timer Recipe
 14. MultiTimer Recipe
 15. Color Detection Recipe
 16. Common Action Recipes
 17. Config + Code Pairs
 18. Patterns to Avoid
 19. Where New Code Belongs
 20. Quick Recipe Index
-

1. How to Use This Library

Most LLARS script logic can be assembled from a few recurring pieces:

1. Read a value from the local `Config.ini`.
2. Randomize a delay or coordinate when appropriate.
3. Perform an action.
4. Log useful events.
5. Repeat through a RunCount loop or a Timer.

6. Let LLARS handle the shared framework lifecycle.

For example, a typical configurable click is only:

```
IniRead, xmin, Config.ini, Action Location, xmin
IniRead, xmax, Config.ini, Action Location, xmax
IniRead, ymin, Config.ini, Action Location, ymin
IniRead, ymax, Config.ini, Action Location, ymax

Random, x, %xmin%, %xmax%
Random, y, %ymin%, %ymax%

NaturalClick(x, y)
```

The matching `Config.ini` is:

```
#Area to click for the action.
[Action Location]
type=coordinate
xmin=
xmax=
ymin=
ymax=
```

That is the basic idea of this library: **show the configuration and the AHK code together whenever possible.** Typed coordinate, hotkey, and color sections power Configuration Status and Reset Config; untyped settings are not reset.

2. Config.ini Building Blocks

Coordinate Area

Use when LLARS should let the user select a rectangular area.

```
#Area the script should click.
[Action Location]
type=coordinate
xmin=
xmax=
ymin=
ymax=
```

Single Pixel Coordinate

Use when the script needs one exact point, commonly for color detection.

```
#Pixel the script should monitor.  
[Pixel Coordinate]  
type=coordinate  
x=  
y=
```

Hotkey

```
#RuneScape hotkey for this action.  
[Action Hotkey]  
type=hotkey  
hotkey=
```

Color

```
#Color that represents the state being monitored.  
[Target Color]  
type=color  
color=0xFFFFFF
```

Random Sleep Range

```
#Random delay before the next action.  
[Sleep Short]  
min=500  
max=1000
```

Optional Timer/Feature

```
#Enable or disable this feature.  
[Example Feature]  
option=false  
min=10000  
max=30000
```

Configuration + Reset Reference

Typed Config.ini metadata connects a script setting to the LLARS Colors/Coordinates/Hotkeys GUI, validation, Configuration Status, Reset Config, and Developer Mode diagnostics.

```
[Action Hotkey]
type=hotkey
hotkey=

[Action Location]
type=coordinate
xmin=
xmax=
ymin=
ymax=

[Target Color]
type=color
color=0xFFFFFF
```

Reset Behavior

- Reset Selected clears the selected typed value only.
- Clear All clears typed hotkey, coordinate, and color values in the current script after confirmation.
- Untyped values, unrelated settings, section names, and the shared LLARS Config.ini are not reset.
- Disabled optional/dependency-disabled sections are Not Required for status/validation, but their saved typed values can still be cleared by Reset Config.

Optional Feature With a Hotkey

```
#Optional action.  
[Example Action]  
option=false  
type=hotkey  
hotkey=
```

Use meaningful section names. The section name appears throughout the script and may be shown to the user during configuration.

3. IniRead Examples

IniRead is one of the most common operations in LLARS scripts.

The general form is:

```
IniRead, OutputVariable, Config.ini, Section Name, KeyName
```

Read One Hotkey

```
IniRead, hk, Config.ini, Action Hotkey, hotkey
```

Read One Coordinate

```
IniRead, x, Config.ini, Pixel Coordinate, x  
IniRead, y, Config.ini, Pixel Coordinate, y
```

Read a Coordinate Area

```
IniRead, xmin, Config.ini, Action Location, xmin  
IniRead, xmax, Config.ini, Action Location, xmax  
IniRead, ymin, Config.ini, Action Location, ymin  
IniRead, ymax, Config.ini, Action Location, ymax
```

Read a Sleep Range

```
IniRead, sa1, Config.ini, Sleep Short, min  
IniRead, sa2, Config.ini, Sleep Short, max
```

Read an Optional Feature

```
IniRead, option, Config.ini, Example Feature, option  
StringLower, option, option
```

The `StringLower` line makes checks such as this predictable:

```
if (option = "true")  
{  
    ; FEATURE CODE  
}
```

Read a Color

```
IniRead, targetColor, Config.ini, Target Color, color
```

4. Random Sleep Examples

Shared LLARS Random Sleep

Random Sleep is configured only in the root LLARS Config.ini [Random Sleep] section. Core reads option, chance, min, and max. When option is not true, the helper returns immediately; do not copy the old block into scripts.

Normal use:

```
; Near the end of the RunCount loop  
LLARS_RandomSleep()  
LLARS_EndLoop()
```

`LLARS_RandomSleep()` rolls the chance when needed, runs the configured sleep only on a successful roll, and handles the shared countdown, counters, GUI state, and logging. It returns the sleep duration, or 0 if none ran.

When the Script Must Know Before the Final Sleep

Only use `LLARS_RandomSleepRoll()` when earlier logic must know whether Random Sleep is coming. The later `LLARS_RandomSleep()` consumes that prepared roll; otherwise call `LLARS_RandomSleep()` directly.

```
LLARS_RandomSleepThisLoop := LLARS_RandomSleepRoll()  
if (LLARS_RandomSleepThisLoop)  
    LLARS_EstimatedSleep(SleepAmount)  
else  
    LLARS_FinalSleep(SleepAmount)  
LLARS_RandomSleep()
```

Basic Compatible Sleep

Creators do not have to use the enhanced estimator. A normal sleep still works and keeps the original predictive runtime-estimation behavior:

```
IniRead, sa1, Config.ini, Sleep Short, min
IniRead, sa2, Config.ini, Sleep Short, max
Random, SleepAmount, %sa1%, %sa2%
Sleep, %SleepAmount%
```

Use this form when the enhanced final-sleep countdown is not needed.

Estimated Sleep Between Two Actions

```
NaturalClick(x, y)

IniRead, sa1, Config.ini, Sleep Short, min
IniRead, sa2, Config.ini, Sleep Short, max
Random, SleepAmount, %sa1%, %sa2%
LLARS_EstimatedSleep(SleepAmount)

Send, {Space}
```

Several Different Sleep Ranges

Config:

```
[Sleep Short]
min=500
max=1000

[Sleep Medium]
min=1500
max=2500

[Sleep Long]
min=4000
max=6000
```

AHK:

```
IniRead, sa1, Config.ini, Sleep Medium, min
IniRead, sa2, Config.ini, Sleep Medium, max
Random, SleepAmount, %sa1%, %sa2%
LLARS_EstimatedSleep(SleepAmount)
```

Change only the section name to use another range. No timer IDs or line numbers are required; use the function name to tell LLARS whether a sleep is ordinary or final.

Final RunCount Sleep

Every RunCount execution path should use LLARS_FinalSleep() for its true final randomized delay. This is the point where LLARS stops predicting the current loop and starts counting down the exact remaining time to the next loop.

```
Random, SleepAmount, %sa1%, %sa2%  
LLARS_FinalSleep(SleepAmount)
```

Use LLARS_EstimatedSleep() when additional loop work follows. Use LLARS_FinalSleep() only at the actual end of that path. If first-run, renewal, or optional branches can change which sleep is last, each branch must identify its own final sleep.

5. Coordinate and Click Examples

LLARS owns coordinate mode. **Do not add your own CoordMode, Mouse, Window or other coordinate-mode override.**

Random Click Inside a Configured Area

```
IniRead, xmin, Config.ini, Action Location, xmin  
IniRead, xmax, Config.ini, Action Location, xmax  
IniRead, ymin, Config.ini, Action Location, ymin  
IniRead, ymax, Config.ini, Action Location, ymax  
  
Random, x, %xmin%, %xmax%  
Random, y, %ymin%, %ymax%  
  
NaturalClick(x, y)
```

Click One Exact Configured Point

For a true point:

```
[Action Point]  
type=coordinate  
x=  
y=
```

AHK:

```
IniRead, x, Config.ini, Action Point, x
IniRead, y, Config.ini, Action Point, y

NaturalClick(x, y)
```

Natural Right Click

Use the same shared helper with "right" for a right-click:

```
NaturalClick(x, y, "right")
```

For normal scripted mouse interaction, prefer `NaturalClick()` instead of raw `Click` or `MouseClick` commands.

Two Different Configured Areas

```
IniRead, xmin, Config.ini, First Location, xmin
IniRead, xmax, Config.ini, First Location, xmax
IniRead, ymin, Config.ini, First Location, ymin
IniRead, ymax, Config.ini, First Location, ymax
Random, x, %xmin%, %xmax%
Random, y, %ymin%, %ymax%
NaturalClick(x, y)
```

```
IniRead, sa1, Config.ini, Sleep Short, min
IniRead, sa2, Config.ini, Sleep Short, max
Random, SleepAmount, %sa1%, %sa2%
Sleep, %SleepAmount%
```

```
IniRead, xmin, Config.ini, Second Location, xmin
IniRead, xmax, Config.ini, Second Location, xmax
IniRead, ymin, Config.ini, Second Location, ymin
IniRead, ymax, Config.ini, Second Location, ymax
Random, x, %xmin%, %xmax%
Random, y, %ymin%, %ymax%
NaturalClick(x, y)
```

Click, Wait, Click

```
IniRead, xmin, Config.ini, First Location, xmin
IniRead, xmax, Config.ini, First Location, xmax
IniRead, ymin, Config.ini, First Location, ymin
IniRead, ymax, Config.ini, First Location, ymax
Random, x, %xmin%, %xmax%
Random, y, %ymin%, %ymax%
NaturalClick(x, y)
```

```
IniRead, sa1, Config.ini, Travel Time, min
IniRead, sa2, Config.ini, Travel Time, max
Random, SleepAmount, %sa1%, %sa2%
Sleep, %SleepAmount%
```

```
IniRead, xmin, Config.ini, Second Location, xmin
IniRead, xmax, Config.ini, Second Location, xmax
IniRead, ymin, Config.ini, Second Location, ymin
IniRead, ymax, Config.ini, Second Location, ymax
Random, x, %xmin%, %xmax%
Random, y, %ymin%, %ymax%
NaturalClick(x, y)
```

6. Hotkey and Send Examples

Send a Configured Hotkey

Config:

```
[Action Hotkey]
type=hotkey
hotkey=
```

AHK:

```
IniRead, hk, Config.ini, Action Hotkey, hotkey
Send, {%hk%}
```

Send Space

```
Send, {Space}
```

Send a Number Key

```
Send, {1}
```

or:

```
Send, {2}
```

Hold and Release a Configured Hotkey

```
IniRead, hk, Config.ini, Action Hotkey, hotkey
```

```
Send, {%hk% down}
```

```
Sleep, 500
```

```
Send, {%hk% up}
```

Hotkey, Wait, Hotkey

```
IniRead, hk, Config.ini, Action Hotkey, hotkey
```

```
Send, {%hk%}
```

```
IniRead, sa1, Config.ini, Sleep Short, min
```

```
IniRead, sa2, Config.ini, Sleep Short, max
```

```
Random, SleepAmount, %sa1%, %sa2%
```

```
Sleep, %SleepAmount%
```

```
Send, {%hk%}
```

7. Color and Pixel Examples

Read a Pixel and Color

Config:

```
[Pixel Coordinate]
```

```
type=coordinate
```

```
x=
```

```
y=
```

```
[Target Color]
```

```
type=color
```

```
color=0xFFFFF
```

AHK:

```
IniRead, targetColor, Config.ini, Target Color, color
IniRead, x, Config.ini, Pixel Coordinate, x
IniRead, y, Config.ini, Pixel Coordinate, y

PixelGetColor, currentColor, %x%, %y%, RGB
```

Do Something When the Color Matches

```
IniRead, targetColor, Config.ini, Target Color, color
IniRead, x, Config.ini, Pixel Coordinate, x
IniRead, y, Config.ini, Pixel Coordinate, y

PixelGetColor, currentColor, %x%, %y%, RGB

if (currentColor = targetColor)
{
    ; ACTION WHEN COLOR MATCHES
}
```

Do Something When the Color Does Not Match

```
PixelGetColor, currentColor, %x%, %y%, RGB

if (currentColor != targetColor)
{
    ; ACTION WHEN COLOR CHANGES
}
```

Prevent Repeated Triggering on the Same Color

This is the pattern used by the Color Detection template:

```

CheckPixel:
if (!LLARS_RUNNING)
    return

IniRead, targetColor, Config.ini, Target Color, color
IniRead, x, Config.ini, Pixel Coordinate, x
IniRead, y, Config.ini, Pixel Coordinate, y

PixelGetColor, currentColor, %x%, %y%, RGB

if (currentColor = targetColor)
{
    SetTimer, CheckPixel, Off

    ; HANDLE THE MATCH HERE

    SetTimer, ResetCheck, 100
}

return

ResetCheck:
if (!LLARS_RUNNING)
    return

IniRead, targetColor, Config.ini, Target Color, color
IniRead, x, Config.ini, Pixel Coordinate, x
IniRead, y, Config.ini, Pixel Coordinate, y

PixelGetColor, currentColor, %x%, %y%, RGB

if (currentColor != targetColor)
{
    SetTimer, ResetCheck, Off
    SetTimer, CheckPixel, 100
}

return

```

8. Optional Feature Examples

Simple Optional Action

Config:

[Optional Action]

option=false

AHK:

```
IniRead, option, Config.ini, Optional Action, option
StringLower, option, option
```

```
if (option = "true")
{
    ; OPTIONAL ACTION
}
```

Optional Hotkey Action

Config:

[Optional Action]

option=false

type=hotkey

hotkey=

AHK:

```
IniRead, option, Config.ini, Optional Action, option
StringLower, option, option
```

```
if (option = "true")
{
    IniRead, hk, Config.ini, Optional Action, hotkey
    Send, {%hk%}
}
```

Optional Timed Action

Config:

[Optional Timer]

option=false

min=10000

max=30000

Startup:

```
IniRead, option, Config.ini, Optional Timer, option  
StringLower, option, option
```

```
if (option = "true")  
{  
    IniRead, sa1, Config.ini, Optional Timer, min  
    IniRead, sa2, Config.ini, Optional Timer, max  
    Random, SleepAmount, %sa1%, %sa2%  
    SetTimer, OptionalTimer, %SleepAmount%  
}
```

Timer:

```
OptionalTimer:  
if (!LLARS_RUNNING)  
    return  
  
; OPTIONAL TIMER ACTION  
  
IniRead, sa1, Config.ini, Optional Timer, min  
IniRead, sa2, Config.ini, Optional Timer, max  
Random, SleepAmount, %sa1%, %sa2%  
SetTimer, OptionalTimer, %SleepAmount%  
  
return
```

Completion:

```
SetTimer, OptionalTimer, Off
```

9. Timer Examples

Start a Repeating Timer Every Second

```
SetTimer, ScriptTimer, 1000
```

Stop a Timer

```
SetTimer, ScriptTimer, Off
```

Schedule a Timer Using a Configured Random Range

```
IniRead, sa1, Config.ini, Action Timer, min
IniRead, sa2, Config.ini, Action Timer, max
Random, SleepAmount, %sa1%, %sa2%
SetTimer, ActionTimer, %SleepAmount%
```

Self-Rescheduling Random Timer

```
ActionTimer:
if (!LLARS_RUNNING)
    return

; ACTION

IniRead, sa1, Config.ini, Action Timer, min
IniRead, sa2, Config.ini, Action Timer, max
Random, SleepAmount, %sa1%, %sa2%
SetTimer, ActionTimer, %SleepAmount%

return
```

This pattern is useful when every interval should be randomized again.

Timer Safety Check

Operational timer labels should normally start with:

```
if (!LLARS_RUNNING)
    return
```

This keeps a timer from performing its action when LLARS is not actively running.

10. Logging Examples

Basic Event

```
Log("ACTION", "Starting banking sequence")
```

Click Event

```
Log("ACTION", "Clicking configured action location")
NaturalClick(x, y)
```

Timer Event

```
Log("TIMER", "Action timer triggered")
```

Optional Feature Event

```
if (option = "true")
{
    Log("OPTION", "Optional action enabled")
}
```

Completion

```
Log("COMPLETE", "Script completed normally")
```

Log useful milestones and decisions. Avoid filling the log with information that will not help debugging.

11. RunCount – Basic Loop Recipe

Use the **Basic Loop** framework when every repetition performs the same sequence.

The template structure is:

```
LLARS_SCRIPT_TYPE := "RunCount"
```

and:

Developer Mode Reference

No creator-facing Developer Mode function is required. Build the script normally and LLARS surfaces the framework information it can observe.

Automatic Creator-Facing Diagnostics

- Typed hotkeys appear under Active Hotkeys.
- Typed hotkey, coordinate, and color problems appear under Configuration Diagnostics.
- Typed colors provide Pixel Target information, including hybrid Timer scripts that also use color detection.
- NaturalClick(), LLARS sleep helpers, lifecycle calls, configuration changes, pixel events, and supported Log() events appear in Recent Framework Actions.
- Script-generated Send key presses are shown automatically; matching typed hotkeys are labeled with their Config.ini section name.

Useful Patterns

```
NaturalClick(x, y)
LLARS_EstimatedSleep(SleepAmount)
LLARS_FinalSleep(SleepAmount)
Log("WAIT", "Random sleep before click: " SleepAmount " ms")
Send, {Space}
```

Use the normal framework patterns above. Do not add diagnostic-only behavior that the script needs in order to function.

Limitation

Developer Mode is a framework-visible diagnostic view, not a full AutoHotkey execution trace. Raw commands that do not pass through observable LLARS behavior may not appear.

```

Start:

if (!LLARS_StartRun())
    return

Loop, %runcount%
{
    LLARS_BeginLoop()

    ; SCRIPT ACTIONS

    LLARS_EndLoop()
}

LLARS_RunComplete()

return

```

Copy/Paste Example: Click + Hotkey + Sleep

Inside the editing markers:

```

Loop, %runcount%
{
    LLARS_BeginLoop()

    IniRead, xmin, Config.ini, Action Location, xmin
    IniRead, xmax, Config.ini, Action Location, xmax
    IniRead, ymin, Config.ini, Action Location, ymin
    IniRead, ymax, Config.ini, Action Location, ymax
    Random, x, %xmin%, %xmax%
    Random, y, %ymin%, %ymax%
    NaturalClick(x, y)

    IniRead, sa1, Config.ini, Sleep Short, min
    IniRead, sa2, Config.ini, Sleep Short, max
    Random, SleepAmount, %sa1%, %sa2%
    Sleep, %SleepAmount%

    IniRead, hk, Config.ini, Action Hotkey, hotkey
    Send, {%hk%}

    LLARS_EndLoop()
}

```

Matching config:

[Action Location]

```
type=coordinate  
xmin=  
xmax=  
ymin=  
ymax=
```

[Action Hotkey]

```
type=hotkey  
hotkey=
```

[Sleep Short]

```
min=500  
max=1000
```

12. RunCount – Advanced Loop Recipe

Use **Advanced Loop** when the first repetition is different.

```
Loop, %runcount%  
{  
    LLARS_BeginLoop()  
  
    if (firstrun = 0)  
    {  
        ; FIRST RUN ONLY  
  
        firstrun := 1  
    }  
    else  
    {  
        ; SECOND RUN AND LATER  
    }  
  
    ; EVERY RUN  
  
    LLARS_EndLoop()  
}
```

Copy/Paste Example: Different First Location

```
Loop, %runcount%
{
    LLARS_BeginLoop()

    if (firstrun = 0)
    {
        IniRead, xmin, Config.ini, First Location, xmin
        IniRead, xmax, Config.ini, First Location, xmax
        IniRead, ymin, Config.ini, First Location, ymin
        IniRead, ymax, Config.ini, First Location, ymax
        Random, x, %xmin%, %xmax%
        Random, y, %ymin%, %ymax%
        NaturalClick(x, y)

        firstrun := 1
    }
    else
    {
        IniRead, xmin, Config.ini, Normal Location, xmin
        IniRead, xmax, Config.ini, Normal Location, xmax
        IniRead, ymin, Config.ini, Normal Location, ymin
        IniRead, ymax, Config.ini, Normal Location, ymax
        Random, x, %xmin%, %xmax%
        Random, y, %ymin%, %ymax%
        NaturalClick(x, y)
    }

    LLARS_EndLoop()
}
```

13. Timer Recipe

Use the Timer template when the user enters an overall run time.

The framework startup is:

```
LLARS_SCRIPT_TYPE := "Timer"
```

and:

Start:

```
if (!LLARS_StartTimerRun())  
    return
```

```
SetTimer, Countdown, 1000
```

```
IfWinNotActive, RuneScape  
{  
    WinActivate, RuneScape  
}
```

```
Gosub, TimerSetup
```

```
return
```

Inside the editing region:

TimerSetup:

```
SetTimer, ScriptTimer, 1000
```

```
return
```

ScriptTimer:

```
if (!LLARS_RUNNING)  
    return
```

```
; REPEATING ACTION
```

```
return
```

Copy/Paste Example: Randomly Scheduled Action

```
TimerSetup:

IniRead, sa1, Config.ini, Action Timer, min
IniRead, sa2, Config.ini, Action Timer, max
Random, SleepAmount, %sa1%, %sa2%
SetTimer, ScriptTimer, %SleepAmount%

return

ScriptTimer:
if (!LLARS_RUNNING)
    return

IniRead, xmin, Config.ini, Action Location, xmin
IniRead, xmax, Config.ini, Action Location, xmax
IniRead, ymin, Config.ini, Action Location, ymin
IniRead, ymax, Config.ini, Action Location, ymax
Random, x, %xmin%, %xmax%
Random, y, %ymin%, %ymax%
NaturalClick(x, y)

IniRead, sa1, Config.ini, Action Timer, min
IniRead, sa2, Config.ini, Action Timer, max
Random, SleepAmount, %sa1%, %sa2%
SetTimer, ScriptTimer, %SleepAmount%

return
```

Make sure completion turns off `ScriptTimer`.

14. MultiTimer Recipe

Use MultiTimer when several actions have independent schedules.

Config:

```
[Timer One]
option=false
min=10000
max=30000

[Timer Two]
option=false
min=30000
max=60000
```

Setup:

```
MultiTimerSetup:

IniRead, option, Config.ini, Timer One, option
StringLower, option, option

if (option = "true")
{
    IniRead, sa1, Config.ini, Timer One, min
    IniRead, sa2, Config.ini, Timer One, max
    Random, SleepAmount, %sa1%, %sa2%
    SetTimer, TimerOne, %SleepAmount%
}

IniRead, option, Config.ini, Timer Two, option
StringLower, option, option

if (option = "true")
{
    IniRead, sa1, Config.ini, Timer Two, min
    IniRead, sa2, Config.ini, Timer Two, max
    Random, SleepAmount, %sa1%, %sa2%
    SetTimer, TimerTwo, %SleepAmount%
}

return
```

Timers:

```

TimerOne:
if (!LLARS_RUNNING)
    return

; TIMER ONE ACTION

IniRead, sa1, Config.ini, Timer One, min
IniRead, sa2, Config.ini, Timer One, max
Random, SleepAmount, %sa1%, %sa2%
SetTimer, TimerOne, %SleepAmount%

return

TimerTwo:
if (!LLARS_RUNNING)
    return

; TIMER TWO ACTION

IniRead, sa1, Config.ini, Timer Two, min
IniRead, sa2, Config.ini, Timer Two, max
Random, SleepAmount, %sa1%, %sa2%
SetTimer, TimerTwo, %SleepAmount%

return

```

Stop function:

```

MultiTimer_StopTimers()
{
    SetTimer, Countdown, Off
    SetTimer, TimerOne, Off
    SetTimer, TimerTwo, Off
}

```

Every new timer must also be added to the stop function.

15. Color Detection Recipe

Use the Color Detection template when the script waits for a visible state.

Config:

```
#Pixel to monitor.
[Pixel Coordinate]
type=coordinate
x=
y=

#Color that triggers the action.
[Target Color]
type=color
color=0xFFFFFF

#Area clicked when the color is detected.
[Action Location]
type=coordinate
xmin=
xmax=
ymin=
ymax=

#Delay before the action.
[Sleep Timer]
min=1
max=1000
```

Detection:

```

CheckPixel:
if (!LLARS_RUNNING)
    return

IniRead, targetColor, Config.ini, Target Color, color
IniRead, x, Config.ini, Pixel Coordinate, x
IniRead, y, Config.ini, Pixel Coordinate, y

PixelGetColor, currentColor, %x%, %y%, RGB

if (currentColor = targetColor)
{
    SetTimer, CheckPixel, Off

    IniRead, sa1, Config.ini, Sleep Timer, min
    IniRead, sa2, Config.ini, Sleep Timer, max
    Random, SleepAmount, %sa1%, %sa2%
    Sleep, %SleepAmount%

    IniRead, xmin, Config.ini, Action Location, xmin
    IniRead, xmax, Config.ini, Action Location, xmax
    IniRead, ymin, Config.ini, Action Location, ymin
    IniRead, ymax, Config.ini, Action Location, ymax
    Random, x, %xmin%, %xmax%
    Random, y, %ymin%, %ymax%
    NaturalClick(x, y)

    SetTimer, ResetCheck, 100
}

return

```

Reset:

```

ResetCheck:
if (!LLARS_RUNNING)
    return

IniRead, targetColor, Config.ini, Target Color, color
IniRead, x, Config.ini, Pixel Coordinate, x
IniRead, y, Config.ini, Pixel Coordinate, y

PixelGetColor, currentColor, %x%, %y%, RGB

if (currentColor != targetColor)
{
    SetTimer, ResetCheck, Off
    SetTimer, CheckPixel, 100
}

return

```

16. Common Action Recipes

Bank Preset Hotkey

Config:

```

[Bank Preset]
type=hotkey
hotkey=

```

AHK:

```

IniRead, hkbank, Config.ini, Bank Preset, hotkey
Send, {%hkbank%}

```

Click an Inventory Area

Config:

```

[Inventory Item]
type=coordinate
xmin=
xmax=
ymin=
ymax=

```

AHK:

```
IniRead, xmin, Config.ini, Inventory Item, xmin
IniRead, xmax, Config.ini, Inventory Item, xmax
IniRead, ymin, Config.ini, Inventory Item, ymin
IniRead, ymax, Config.ini, Inventory Item, ymax
Random, x, %xmin%, %xmax%
Random, y, %ymin%, %ymax%
NaturalClick(x, y)
```

Click Then Press Space

```
NaturalClick(x, y)

IniRead, sa1, Config.ini, Sleep Short, min
IniRead, sa2, Config.ini, Sleep Short, max
Random, SleepAmount, %sa1%, %sa2%
Sleep, %SleepAmount%

Send, {Space}
```

Optional Prayer/Ability Style Hotkey

Config:

```
[Prayer]
option=false
type=hotkey
hotkey=
```

AHK:

```
IniRead, option, Config.ini, Prayer, option
StringLower, option, option

if (option = "true")
{
    IniRead, hk, Config.ini, Prayer, hotkey
    Send, {%hk%}
}
```

Random Timer for an Anti-AFK Style Action

Config:

[AFK]

min=252000

max=294000

Startup:

```
IniRead, sa1, Config.ini, AFK, min
IniRead, sa2, Config.ini, AFK, max
Random, SleepAmount, %sa1%, %sa2%
SetTimer, AntiAFK, %SleepAmount%
```

Timer:

```
AntiAFK:
if (!LLARS_RUNNING)
    return

; ANTI-AFK ACTION

IniRead, sa1, Config.ini, AFK, min
IniRead, sa2, Config.ini, AFK, max
Random, SleepAmount, %sa1%, %sa2%
SetTimer, AntiAFK, %SleepAmount%

return
```

17. Config + Code Pairs

This section is designed for direct copy/paste.

Pair A – Randomized Click

Config:

```
[Action Location]
type=coordinate
xmin=
xmax=
ymin=
ymax=
```

AHK:

```
IniRead, xmin, Config.ini, Action Location, xmin
IniRead, xmax, Config.ini, Action Location, xmax
IniRead, ymin, Config.ini, Action Location, ymin
IniRead, ymax, Config.ini, Action Location, ymax
Random, x, %xmin%, %xmax%
Random, y, %ymin%, %ymax%
NaturalClick(x, y)
```

Pair B – Hotkey

Config:

```
[Action Hotkey]
type=hotkey
hotkey=
```

AHK:

```
IniRead, hk, Config.ini, Action Hotkey, hotkey
Send, {%hk%}
```

Pair C - Shared Random Sleep

LLARS Config.ini:

```
[Random Sleep]
option=false
chance=3
min=30000
max=285000
```

AHK:

```
LLARS_RandomSleep()
```

Pair D – Pixel Color

Config:

[Pixel Coordinate]

type=coordinate

x=

y=

[Target Color]

type=color

color=0xFFFFFF

AHK:

```
IniRead, targetColor, Config.ini, Target Color, color
IniRead, x, Config.ini, Pixel Coordinate, x
IniRead, y, Config.ini, Pixel Coordinate, y
PixelGetColor, currentColor, %x%, %y%, RGB

if (currentColor = targetColor)
{
    ; ACTION
}
```

Pair E – Optional Hotkey

Config:

[Optional Action]

option=false

type=hotkey

hotkey=

AHK:

```
IniRead, option, Config.ini, Optional Action, option
StringLower, option, option

if (option = "true")
{
    IniRead, hk, Config.ini, Optional Action, hotkey
    Send, {%hk%}
}
```

Pair F – Random Recurring Timer

Config:

```
[Action Timer]
```

```
min=10000
```

```
max=30000
```

Startup:

```
IniRead, sa1, Config.ini, Action Timer, min
```

```
IniRead, sa2, Config.ini, Action Timer, max
```

```
Random, SleepAmount, %sa1%, %sa2%
```

```
SetTimer, ActionTimer, %SleepAmount%
```

Timer:

```
ActionTimer:
```

```
if (!LLARS_RUNNING)
```

```
    return
```

```
; ACTION
```

```
IniRead, sa1, Config.ini, Action Timer, min
```

```
IniRead, sa2, Config.ini, Action Timer, max
```

```
Random, SleepAmount, %sa1%, %sa2%
```

```
SetTimer, ActionTimer, %SleepAmount%
```

```
return
```

18. Patterns to Avoid

Do Not Override Coordinate Mode

Do not add:

```
CoordMode, Mouse, Window
```

or another script-specific coordinate mode unless the LLARS framework itself is intentionally being changed.

Do Not Copy NaturalClick Into the Script

Use the shared helper:

```
NaturalClick(x, y)
```

```
NaturalClick(x, y, "right")
```

Do not create another generic natural-click function, and do not replace normal scripted clicks with raw `Click` or `MouseClick` commands.

Do Not Use EstimatedSleep for the Final Delay

Normal in-loop delay:

LLARS_EstimatedSleep(SleepAmount)

True final delay on that execution path:

LLARS_FinalSleep(SleepAmount)

The final call lets LLARS replace the prediction with the exact randomized countdown to the next loop. For conditional endings, make sure every possible path reaches the correct LLARS_FinalSleep().

Do Not Rebuild the Main GUI

Use the selected LLARS template. Core already owns the common GUI, Configuration Status, and Reset Config.

Do Not Rebuild RunCount State

For RunCount scripts, use:

```
LLARS_StartRun()  
LLARS_BeginLoop()  
LLARS_EndLoop()  
LLARS_RunComplete()
```

Do Not Rebuild Timer Startup

For Timer/MultiTimer scripts, use:

```
LLARS_StartTimerRun()
```

Do Not Remove the Script Editing Markers

Keep exactly:

```
; =====  
; SCRIPT_EDIT_BEGIN_4C4C415253  
; =====
```

and:

```
; =====  
; SCRIPT_EDIT_END_4C4C415253  
; =====
```

Do Not Use AutoHotkey v2 Syntax

LLARS currently requires:

```
#Requires AutoHotkey v1.1.37.02
```

Use the established AHK v1 command style shown in the templates and this library.

19. Where New Code Belongs

Basic RunCount

Put the repeated automation inside:

```
Loop, %runcount%  
{  
    LLARS_BeginLoop()  
  
    ; YOUR CODE  
  
    LLARS_EndLoop()  
}
```

Advanced RunCount

Put first-run differences inside:

```
if (firstrun = 0)  
{  
    ; FIRST RUN  
    firstrun := 1  
}  
else  
{  
    ; LATER RUNS  
}
```

Timer

Use:

```
TimerSetup:
```

for one-time scheduling/setup and:

```
ScriptTimer:
```

for repeating timer behavior.

MultiTimer

Use:

```
MultiTimerSetup:
```

to schedule independent timers, then give each timer its own label.

Remember to stop each one in:

```
MultiTimer_StopTimers()
```

Color Detection

Use:

```
CheckPixel:
```

to detect the configured state and:

```
ResetCheck:
```

to wait until that state changes before allowing another trigger.

20. Quick Recipe Index

If you need to...

Click a configurable area: use **Random Click Inside a Configured Area**.

Press a configurable RuneScape key: use **Send a Configured Hotkey**.

Use the enhanced RunCount sleep estimate: use **Preferred RunCount Sleep**.

Wait without the enhanced final-sleep countdown: use **Basic Compatible Sleep**.

Make a feature optional: use **Simple Optional Action** or **Optional Hotkey Action**.

Run an action every random interval: use **Self-Rescheduling Random Timer**.

React to a screen color: use **Prevent Repeated Triggering on the Same Color**.

Repeat the same sequence N times: start from **RunCount – Basic Loop**.

Make the first run different: start from **RunCount – Advanced Loop**.

Run for a user-selected amount of time: start from **Timer Recipe**.

Run several independent timed actions: start from **MultiTimer Recipe**.

Watch pixel colors and react: use **Color Detection Recipe** or **Multi Color Detection Recipe** (page 41).

Final Rule

When building a LLARS script, the easiest approach is:

Template + Config section + small copy/paste recipe + task-specific behavior.

Do not rebuild the framework.

Use the framework for the shared systems, use `Config.ini` for user-configurable values, and keep the script focused on the actual RuneScape automation it was created to perform.

21. Standard Creator API

The preferred creator API keeps scripts focused on task-specific automation while LLARS handles repetitive configuration, timing, click, status, and diagnostic behavior.

Config helper functions

Use these helpers instead of repeating IniRead and validation boilerplate throughout a script. They use the current script Config.ini by default; pass "shared" only when a script intentionally needs LLARS Config.ini.

```
value := LLARS_ConfigRead("Section", "key", "default")
name := LLARS_ConfigReadString("Section", "name", "", true)
enabled := LLARS_ConfigReadBool("Feature")
count := LLARS_ConfigReadInteger("Settings", "count", "", 1)
delay := LLARS_ConfigReadNumber("Settings", "delay", "", 0)
hotkey := LLARS_ConfigReadHotkey("Action Hotkey")
color := LLARS_ConfigReadColor("Target Color")
LLARS_ConfigReadRange("Sleep Short", minValue, maxValue)
LLARS_ConfigReadPoint("Action Location", x, y)
```

These helpers do not create a second validation system. Invalid helper reads use the existing CONFIG ERROR path, while LLARS startup validation and Developer Mode Configuration Diagnostics remain the authoritative checks.

Action and timing helpers

```
LLARS_Click("Action Location")
LLARS_Sleep("Sleep Short")
LLARS_Sleep("Final Timer", true)
interval := LLARS_TimerInterval("Timer One")
LLARS_SetStatus("Banking")
if LLARS_PixelMatches("Pixel Coordinate", "Target Color")
    ; HANDLE MATCH
LLARS_RandomSleep()
```

Prefer these shared calls for new scripts. Existing low-level helpers remain available for unusual cases, but creators should not rebuild shared LLARS behavior inside each script.

Safety behavior

LLARS_Click() returns false after the LLARS run ends. LLARS_PixelMatches() returns false unless RuneScape is active; when using a negative/reset check, return while RuneScape is inactive before treating false as a state change.

22. Developer Mode and Lightweight Mode

Full Developer Mode and Lightweight Mode now use the same authoritative Recent Framework Actions history. Lightweight Mode is only a compact view of the newest entries; it does not maintain a separate filtered log.

Recent Framework Actions

- LLARS control hotkeys such as Start, Pause, Resume, Exit, and Developer Mode are recorded as Pressed and Released events when triggered by hotkey.
- Script-generated configured hotkeys and ordinary generated keys are recorded as paired Pressed and Released events.
- Framework clicks, sleeps, status changes, lifecycle events, timers, Random Sleep, pixel events, and configuration activity use the same event stream.
- Repeated generated key-down noise is suppressed until the matching release so the history stays readable.
- Unrelated physical keyboard/mouse input and driver-generated thumb-button input are not treated as framework actions.

Mouse / Pixel Live Inspector

The live inspector only reads coordinates and pixel color while a RuneScape window is the active window. If RuneScape is open but not active, the inspector reports INACTIVE and shows no live X/Y/RGB values. If no RuneScape window is found, it reports NOT FOUND.

This prevents the Developer Mode inspector from showing coordinates or colors from another application after an Alt-Tab.

Configuration diagnostics

No separate creator validation mode is required. LLARS already blocks a run when required configuration is invalid or missing, and Developer Mode shows active Config.ini problems under Configuration Diagnostics. Keep those systems as the single validation source of truth.

23. Creator API Cheat Sheet

Use this page as the quick reference when building a new LLARS script. Start from Templates\Script Generator.ahk or the closest template, then keep task-specific behavior inside the script editing markers.

Configuration

```
LLARS_ConfigRead("Section", "key", default)
LLARS_ConfigReadString("Section", "key", default, required)
LLARS_ConfigReadBool("Section")
LLARS_ConfigReadInteger("Section", "key", default, min, max)
LLARS_ConfigReadNumber("Section", "key", default, min, max)
LLARS_ConfigReadHotkey("Action Hotkey")
LLARS_ConfigReadColor("Target Color")
LLARS_ConfigEnabled("Optional Feature")
LLARS_ConfigReadRange("Timer", minValue, maxValue)
LLARS_ConfigReadPoint("Location", x, y)
```

Common actions

```
LLARS_Click("Location")
LLARS_Click("Location", "right")
LLARS_Sleep("Sleep Short")
LLARS_Sleep("Final Timer", true) ; final RunCount sleep
interval := LLARS_TimerInterval("Timer One")
LLARS_RandomSleep()
LLARS_SetStatus("Running", "Current Activity")
LLARS_PixelMatches("Pixel Coordinate", "Target Color")
```

Run lifecycle

```
LLARS_StartRun()          LLARS_StartTimerRun()
LLARS_BeginLoop()         LLARS_EndTimerRun()
LLARS_EndLoop()
LLARS_RunComplete()
```

Rule of thumb: if LLARS already owns the behavior, call the shared API instead of copying its implementation into the script. Keep Config.ini user-facing, keep validation centralized, and keep the script focused on the RuneScape task itself.

24. Multi Color Detection Recipe

Use the Multi Color Detection template when one watched pixel can represent several valid states and each completed trigger should advance to the next configured click location.

Core pattern

```
MultiColorColorSections := ["Target Color One", "Target Color Two"]
MultiColorLocationSections := ["Action Location One", "Action Location Two"]

MatchedColorSection := MultiColor_FindMatchedColor()
if (MatchedColorSection != "")
{
    SetTimer, CheckPixel, Off
    LLARS_Sleep("Sleep Timer")

    if (!LLARS_RUNNING)
        return

    MultiColor_ClickNextLocation()
    SetTimer, ResetCheck, 100
}
```

How the reset works

After a match, detection stays disabled until the watched pixel no longer matches any configured target color. This prevents one unchanged state from triggering repeatedly. The next completed trigger uses the next location and wraps back to the first location after the list ends.

Config sections

```
[Pixel Coordinate]
type=coordinate
x=
y=

[Target Color One]
type=color
color=0xFFFFFF

[Action Location One]
type=coordinate
xmin=
xmax=
ymin=
ymax=
```

Add Target Color Two, Target Color Three, and additional Action Location sections as needed. Then add those exact section names to MultiColorColorSections and MultiColorLocationSections. LLARS_PixelMatches() only samples while RuneScape is active.
